

动态规划入门

教学计划

- ??动态规划是什么??
- 了解动态规划的几个简单的例子
- 动态规划是什么
- 认识常见的动态规划模型

前言

出于一些众所周知的原因今年仍然由本人来讲动态规划这一节的课程. 根据大家的反馈, 去年的课程设计的比较困难, 使得主讲人和听众在信息传达上出现了鸿沟: 大家不明白我在讲什么, 我也不知道大家不明白哪里. 这是课程设计的失败, 在此深表歉意. 今年的内容吸取了去年的教训以及不少前辈们的意见, 进行了一定幅度的重写, 若有不妥也欢迎诸位听众指正.

??动态规划是什么??

动态规划(Dynamic Programming, 简称DP)是一种用于解决具有最优子结构问题的算法. 它通过将问题分解为子问题, 并保存子问题的解来避免重复计算, 从而提高效率.

动态规划的核心思想是将一个复杂问题分解为多个简单的子问题, 并通过保存子问题的解来避免重复计算. 具体来说, 动态规划通常包括以下三个步骤:

1. **建立状态转移方程**: 状态转移方程描述了如何通过子问题的解来构建原问题的解. 例如, 斐波那契数列的状态转移方程为 $f(n) = f(n-1) + f(n-2)$.
2. **缓存并复用以往结果**: 通过缓存子问题的解, 可以避免重复计算, 从而提高算法效率. 例如, 在计算斐波那契数列时, 可以将每个子问题的解存储在数组中, 以便后续使用.
3. **按顺序从小往大算**: 按照问题规模从小到大的顺序依次计算子问题的解, 最终得到原问题的解. 例如, 在计算斐波那契数列时, 可以从 $f(0)$ 和 $f(1)$ 开始, 依次计算到 $f(n)$.

这里面我们涉及到了状态, 状态转移方程这些概念, 在稍后的例子中我们会为大家解释. 以上应该是大家在绝大多数算法书籍里头看到的对动态规划的介绍.

动态规划的几个简单的例子

计算斐波那契数列的第 n 项

众所周知斐波那契数列 $\langle \text{fib}_i : i \in \mathbb{N} \rangle$ 是满足如下条件的唯一数列

1. $\text{fib}_0=0, \text{fib}_1=1$.

2. 对于 $i \in \mathbb{N}$, $\text{fib}_{i+2} = \text{fib}_{i+1} + \text{fib}_i$.

请你写一个程序, 它接受一个输入 $n (\leq 10^6)$, 输出斐波那契数列第 n 项模 $1e9+7$ 的值. 首先我们可以写一个暴力求值的程序:

```
const int mo=1e9+7;
int calc(int n){
    if(n==0)return 0;
    if(n==1)return 1;
    return (calc(n-1)+calc(n-2))%mo;
}
signed main(){
    int n;
    cin>>n;
    cout<<calc(n)<<endl;
}
```

在这个程序中, 向函数 `calc()` 输入值 n , 它会根据斐波那契的构造方式来递归地计算我们想要的第 n 项值. 之后我们来学习一种优化这种暴力搜索的优化方法: **记忆化搜索**.

每次调用 `calc(n)` 的时候, 我们就需要调用 `calc(n-1)` 和 `calc(n-2)`, 之后 `calc(n-1)` 自己又去调用 `calc(n-2)` ... 我们发现一个问题, 几乎每个 `calc(i)` 都被重复调用太多次了! 明明只用调用过一次 `calc(i)` 我们就可以知道第 i 项的值了, 那有没有办法把这个结果在第一次被计算出来的时候就记下来, 下次再调用的时候直接不用去计算而是直接把记下来的值返回呢? 这就是记忆化搜索.

```
const int mo=1e9+7, N=1e6+5;
int f[N];
int calc(int n){
    if(n==0)return 0;
    if(n==1)return 1;
    if(f[n]==-1){
        f[n]=(calc(n-1)+calc(n-2))%mo;
    }
    return f[n];
}
signed main(){
    memset(f, -1, sizeof(f));
    int n;
    cin>>n;
    cout<<calc(n)<<endl;
}
```

最后我们来介绍一下动态规划的做法, 我们首先需要设计出子问题出来, 为了解决问题 (n) : " fib_n 的值是多少". 我们需要去解决问题 $(n-1)$: " fib_{n-1} 的值是多少". 和问题 $(n-2)$: " fib_{n-2} 的值是多少". 在这里头问题 $(n-1)$, $(n-2)$ 是 (n) 的子问题, 更一般的来说, (n) 的子问题是 (i) : " $\text{fib}_i (i < n)$ 的值是多少". 于是我们设计状态: 令数组 `f[]` 的第 i 项 `f[i]` 表示问题 (i) 的答案, 那

么通过问题 (i-2), (i-1) 的答案我们可以解答问题 (i), 并且将得到的答案记录到 f[i] 里面, 具体的代码如下.

```
const int mo=1e9+7,N=1e6+5;
int f[N],n;
signed main(){
    cin>>n;
    f[0]=0,f[1]=1;
    for(int i=2;i<=n;++i){
        f[i]=(f[i-1]+f[i-2])%mo;
    }
    cout<<f[n]<<endl;
}
```

不过需要注意问题的求解顺序! 比如我要求解问题 (i), 但是如果这个时候问题 (i-1) 和问题 (i-2) 都还没被求解, 那么我们设计的求解方法就会失效, 因此为了保证求解问题 (i) 时问题 (i-1), (i-2) 都已经被求解过了, 我们采取从小到大来求解问题的顺序.

计算格路方案数

你有一个 N 行 M 列的网格图, 一开始你在最左下角的格子里, 每次移动你可以选择移动到你当前所处的格子的右边一格或者上面一格, 问你走到网格的最右上角的格子有多少种不同的路线.

你需要写一个程序, 接受输入 N,M($N,M \leq 10^3$), 并输出答案对 $1e9+7$ 取模的结果.

在此我们介绍动态规划的解法. 我们先在网格上建立平面直角坐标从而方便去描述我们格子的位置, 令左下角的坐标为 (1,1), 右上角的坐标为 (N,M). 令问题 [i|j] 为 "从起点 (1,1) 出发, 有多少种路线能够走到格子 (i,j)", 之后我们设计状态: 令二维数组 f[] 的第 (x,y) 项 f[x,y] 表示问题 [x|y] 的答案.

接下来我们来尝试求解问题 [i|j], 首先请大家思考: 对于一条从 (1,1) 走到 (i,j) 的路线, 在最终走到 (i,j) 这个格子前一步我们会有可能位于哪些格子上呢? 是 (i-1,j) 和 (i,j-1)! 事实上, 所有最终走到 (i,j) 这个格子上的路径要么是以 (i-1,j) 为终点的路径再向上走一步, 要么是以 (i,j-1) 为终点的路径再向右走一步, 所以问题 [i|j] 的答案就是 [i-1|j] 与 [i|j-1] 的答案之和.

最后, 我们需要注意答案求解的顺序, 这里头我们采取从小到大遍历 i, 之后从小到大遍历 j 的顺序来依次解答问题 [i|j], 于是我们可以写如下的程序

```
const int mo=1e9+7,N=1005;
int f[N][N],n,m;
signed main(){
    cin>>n>>m;
    f[1][1]=1;
    for(int i=1;i<=n;++i){
        for(int j=1;j<=m;++j){
            if(i==1&&j==1)continue; //[1|1] 已经被初始化了
            f[i][j]=0;
            if(i>1)f[i][j]+=f[i-1][j];
            if(j>1)f[i][j]+=f[i][j-1];
        }
    }
}
```

```

        f[i][j]%=mo;
    }
}
cout<<f[n][m]<<endl;
}

```

当然,熟悉组合数学的同学可以一眼就看出来问题 $[i][j]$ 的答案就是 $\binom{i+j}{i}$, 因此可以从组合数学的角度进一步来验证我们这个递推关系的正确性.

补充

我们来看看记忆化搜索的代码. 我们先写一个暴力搜索的代码.

```

int dfs(int x,int y){
    if(x==1&&y==1)return 1;
    int ans=0;
    if(x>1)ans+=dfs(x-1,y);
    if(y>1)ans+=dfs(x,y-1);
    return ans%mo;
}

```

之后我们按照记忆化搜索的方法, 把每个 (x,y) 都记录下来, 于是就有

```

int dfs(int x,int y){
    if(f[x][y]!=-1)return f[x][y];
    if(x==1&&y==1)return 1;
    f[x][y]=0;
    if(x>1)f[x][y]+=dfs(x-1,y);
    if(y>1)f[x][y]+=dfs(x,y-1);
    f[x][y]%=mo;
    return f[x][y];
}

```

计算格路最大权路径

你有一个 N 行 M 列的网格图, 每个网格内都有若干个金币, 一开始你在 $(1,1)$, 每次可以从 (i,j) 走到 $(i+1,j)$ 或者 $(i,j+1)$, 最终你要走到 (N,M) , 在路途中你会把沿路走过的格子金币都采集起来. 请你设计一条路径使得最后能够拿到最多的金币.

你需要设计一个程序, 第一行接受输入 $N, M(N, M \leq 10^3)$; 之后的 N 行每行接受 M 个输入, 其中第 i 行的第 j 个输入表示格子 (i,j) 里有多少枚金币; 最后输出一个数表示答案.

用一个二维数组 $a[]$ 来记录每个格子里的金币的数量: 格子 (x,y) 里头有 $a[x,y]$ 个的金币. 之后依照惯例我们设计问题 $[i][j]$ 为 "在所有以 (i,j) 为终点的路径中, 能够采集到的最多的金币是多少", 之后设计状态 $f[x,y]$ 表示问题 $[x][y]$ 的答案.

之后我们尝试回答问题 $[i|j]$, 根据我们上一个题目的经验, 我们把所有以 (i,j) 为终点的路径划分成两类 A,B. 其中 A 表示经过 $(i-1,j)$ 的路径, B 表示经过 $(i,j-1)$ 的路径, 为了求出 $[i|j]$ 的答案, 我们分别需要求出 A 中的路径的最大值和 B 中的路径的最大值, 再在这两者之间取最大值. 那么 A 中的那个最大值路径该怎么选呢? 就是以 $(i-1,j)$ 为终点的最大值路径在往上走一格, 于是我们就需要用到问题 $[i-1|j]$ 的答案, 这个路径的权值即为 $f[i-1,j]+a[i,j]$; 类似的 B 类的最大值即为 $f[i,j-1]+a[i,j]$. 因此只要知道了 $[i-1|j]$ 和 $[i|j-1]$ 的答案我们就可以求解出 $[i|j]$ 了. 于是可以设计代码如下

```
const int N=1e3+5;
int n,m,a[N][N],f[N][N];
signed main(){
    cin>>n>>m;
    for(int i=1;i<=n;++i){
        for(int j=1;j<=m;++j){
            cin>>a[i][j];
        }
    }
    f[1][1]=a[1][1];
    for(int i=1;i<=n;++i){
        for(int j=1;j<=m;++j){
            if(i==1&&j==1)continue;
            int tmp=0;
            if(i>1)tmp=max(tmp,a[i][j]+f[i-1][j]);
            if(j>1)tmp=max(tmp,a[i][j]+f[i][j-1]);
            f[i][j]=tmp;
        }
    }
}
```

动态规划是什么

从归约来看动态规划

本部分可以被看作一个大型的闲话段落, 不包含任何技巧技术的教学, 仅为本人对于动态规划思想的一些总结. 当然我也不打算通过某种类形式化的语言来准确描述什么是动态规划, 说到底其实是我不能, 因为这终究涉及到了一些存在论的哲学问题, 我们的形式语言的范式在描述这类问题上是有本源性的缺陷的. 所以接下来的一些术语大家可以放在自然语言中去理解(比如关心某个东西为什么是集合而不是真类等等)而不必过分追求准确性.

在上述的三个例子里, 我们关注到动态规划最关键的地方是要利用子问题的答案来求解原问题, 在递归论/可计算性理论里面, 我们把这种通过一个问题(判定数 x 是否属于集合 P)来解决另一个问题(判定数 x 是否属于集合 Q)的行为成为归约, 当然在这里我们不会展开归约和图灵度的相关知识. 用类似的术语来说, 动态规划是在通过把问题归约到子问题上的方式来计算原问题: 我们不知道一个问题的答案, 但是如果我们知道了一些其他问题的答案, 那么我们就可以通过某些办法来计算出这个问题的答案.

那么何谓原问题与子问题?是不是规模小的问题就是规模大的问题的子问题呢?大多数情况下是,但是我打算在此给一个更加精细的解答.比如在上一节里头我们举的三个例子,我们把每个例子里头的问题([i],[i|j])都抽象出来变成点,假定它们组成的集合是 S ,称之问题空间,那么对于每个问题节点 x ,我们可以找到 S 的某个子集 P_x ,使得通过某个可计算的过程即可求的 x 的答案.而动态规划就是把 S 中的所有点都放在一个序列 $\langle \alpha_i \rangle$ 里,使得对于全部的 i 都有 $P_{\alpha_i} \subseteq \{\alpha_0, \alpha_1, \dots, \alpha_{i-1}\}$.在这个过程中,我们要注意每个问题节点都是独立的,封装好的,我们的可计算过程只能从外部去调用这个节点的结果,而不能把这个节点拆开去修改,查看内部的信息,这是动态规划的"无后效性".

我们再来回答何谓"可计算的",这是递归论里的基础问题,学习过离散数学的同学就会很清楚,可计算的就是递归的函数(注意不是半递归的,因为半递归的可能会不停机).如果用自然语言来描述它,那么它最大的特点就是一定要在有穷的步骤里头得到结果.在结尾的 **bonus** 处我会给出一些问题空间里出现环型结构的情形,这种情况下我们是无法用动态规划来求解的(因为我们可以一直在环上转圈,从而逾越出**有穷**的范畴),需要重新设计.

之后我们来谈一谈动态规划和记忆化搜索的关系,在前面的三个例子中我们的动态规划都是采用 **for** 循环来实现的.但是正如我们前面所言,动态规划本质上是问题间的归约(递归),于是实际上更加自然的实现我们的归约过程的方法是递归.事实上,记忆化搜索也是动态规划的实现方法之一.具体可以参见如下的伪代码

```
vis[]=false, f[]
dfs(n):                                求解问题节点 n
    if vis[n]=true :                  如果我们已经知道了问题节点 n 的答案
        return f[n]
    for x in P_n :                     遍历 P_n
        calc(f[n], dfs(x))           调用我们设计的计算过程
    vis[n]=true
    return f[n]
```

综上所述,动态规划可以理解为如下的步骤:

1. 设计问题空间 S .
2. 设计这个可计算过程(并证明这个可计算过程是正确的!),并给每个问题空间内的点 x 找到它的 P_x .
3. 设计求解顺序(使用记忆化搜索就不需要这一步).

认识常见的动态规划模型

- 序列型动态规划
- 平面型动态规划
- 背包问题
- 树上动态规划
- 子集上的动态规划

接下来我们将略去代码实现, 而着重讲解问题空间, 归约集合与计算过程的设计. 本节的作业就是实现我们课上讲的题目.

序列型动态规划

最长上升子序列

给定一个长度为 N 的正整数序列 $\langle a_i \rangle$, 请你计算最长的严格递增的子序列的长度.

一个子序列本质上就是下标集 $\{1, 2, \dots, N\}$ 的子集 S , 于是我们考虑从小到大向子序列的下标集添加元素. 具体来讲, 设计问题 $[i]$ 为 "下标集中的最大元素恰好为 i 的最长上升子序列的长度是多少". 最后原问题的答案就是 $\max_{i \leq N} f[i]$.

对于问题 $[i]$, 我们考虑在一个下标集的 S 中, 把最大值 i 去掉后, 最大值的是谁(其实就是这样的上升子序列中 a_i 的前一个数是谁)? 它一定是某个满足 $j < i$ 且 $a_j < a_i$ 的 j . 如果我们固定了这样的 j , 为了使得这个子序列的长度最大, 我们一定会选取以 a_j 为结尾的最长的上升子序列之后再拼上 a_i , 于是我们就会需要 $[j]$ 的答案了. 总结一下我们的计算流程, 将所有以 a_i 为结尾的上升子序列按照 j 分类, 我们只需要对每一类求得最大值(即 $[j]$) 之后对所有类取最大值即可. 于是对于每个 i 而言 $P[i]$ 就是 $\{[j] : j < i \text{ 且 } a_j < a_i\}$. 伪代码如下

```
for 1<=i<=n:
    f[i]=1
    for j<i:
        if a[j]<a[i]:
            f[i]=max(f[i], f[j]+1)
```

时间复杂度为 $O(N^2)$

最长公共子序列

给定两个长度为 N 的正整数序列 $\langle a_i \rangle, \langle b_i \rangle$, 求最长的公共子序列的长度.

设计问题 $[x|y]$ 为 "序列 $\langle a_1, a_2, \dots, a_x \rangle, \langle b_1, b_2, \dots, b_y \rangle$ 的最长公共子序列的长度". 最后原问题的答案为 $f[N|N]$.

为了解决 $[x|y]$, 我们考虑把这两个序列的前缀的所有公共子序列分成如下的三类

1. a_x, b_y 同时出现在公共子序列中(这个时候它们两个都是子序列的最后一个元素, 所以应该有 $a_x = b_y$)
2. a_x 不出现在公共子序列中
3. b_y 不出现在公共子序列中

注意到 2, 3类可能是有重叠的, 但是我们求的是最优值, 所以只需要保证每种可能的情况都被计算到就行了(计数类的要求不重不漏的统计到每种情况). 对于第 1 类, 在满足了 $a_x = b_y$ 后, 我们需要在前缀 $\langle a_1, a_2, \dots, a_{x-1} \rangle, \langle b_1, b_2, \dots, b_{y-1} \rangle$ 中找到最长的公共子序列后, 再把 a_x, b_y 拼接在后面, 于是取 $[x-1|y-1]$ 的答案再 +1 即可; 对于第 2 类, a_x 不在公共子序列中的情

况就是前缀 $\langle a_1, \dots, a_{x-1} \rangle$ 和 $\langle b_1, \dots, b_y \rangle$ 的答案, 即 $[x-1|y]$; 第 3 类同理即为 $[x|y-1]$. 因此对于问题 $[x|y]$ 我们只需要知道 $[x-1|y-1], [x-1|y], [x|y-1]$ 即可计算答案.

时间复杂度 $O(N^2)$

最长公共上升子序列(作业挑战题)

给定两个长度为 N 的正整数序列 $\langle a_i \rangle, \langle b_i \rangle$, 求最长的公共上升子序列的长度, 设计一个时间复杂度不超过 $O(N^3)$ 的算法.

- Hint: 考虑设计问题 $[x|y]$ 为 "前缀 $\langle a_1, \dots, a_x \rangle$ 与 $\langle b_1, \dots, b_y \rangle$ 的以 b_y 为结尾的最长公共上升子序列的长度".

平面型动态规划

方格取数

设有 N 行 M 列的方格图, 每个方格中都有一个整数. 现有一只小熊, 想从图的左上角走到右下角, 每一步只能向上, 向下或向右走一格, 并且不能重复经过已经走过的方格, 也不能走出边界. 小熊会取走所有经过的方格中的整数, 求它能取到的整数之和的最大值. 设计一个 $O(N^2M)$ 复杂度的算法.

- Hint: 大家可以画一画小熊走的路应该是长什么样的?

形式化的来讲, 小熊走的一条完整的路径可以被切分成 M 段, 其中第 i 段是在第 i 列上连续的若干行, 并且第 i 列的终点的行数必须等于第 $i+1$ 列的起点的行数. 于是我们设计问题 $[i|j]$ 为 "在所有终点为 (i, j) 的小熊可以走的路径中, 路径权值和最大是多少". 最后原问题的答案就是 $f[M|N]$.

为了解决问题 $[i|j]$, 我们考虑把所有以 (i, j) 为终点的路径按照第 i 列的开头的行数来分类, 假定是第 k 行, 那么为取得这一类的最大值, 我们需要使得在第 i 列以前的路径尽量大, 并且由于起点是第 k 行, 那么根据限制上一列的终点也要是第 k 行, 所以这一类的答案就是 $f[i-1|k] + \sum_{x \in [j, k]} a[i, x]$. 为了方便计算这个和式我们可以采取前缀和的优化技术: 我们一开始就令数组 $sum[i, j] = \sum_{k \leq j} a[i, k]$, 于是 $\sum_{x \in [j, k]} a[i, x]$ 就可以用 $sum[i, \max(j, k)] - sum[i, \min(j, k) - 1]$ 来计算了. 总结一下为了计算问题 $[i|j]$, 我们对每个 k 并求出 $f[i-1|k] + \sum_{x \in [j, k]} a[i, x]$, 并取出里头的最大值即可. 时间复杂度是 $O(N^2M)$.

方格取数plus

同样的问题, 给出一个 $O(NM)$ 的算法.

- Hint: 有没有更好的方法来刻画小熊的路径呢? 用更通俗的话来解释 "第 i 列的终点的行数必须等于第 $i+1$ 列的起点的行数" 这个限制.

我们注意到小熊每一列的路径都是有方向的(从下到上/从上到下), 于是我们可以把方向写进我们的问题里, 令问题 $[x|y|0/1]$ 表示 "在所有终点为 (i, j) 的且第 i 列的方向为 0 :" 从下到上"/ 1 :" 从上到下"的小熊可以走的路径中, 路径权值和最大是多少". 最后原问题的答案就是 $f[M|N|1]$

对于问题 $[i|j|0]$, 显然它前一步要么是从下面来的, 要么是从前一行来的. 如果是从下面来的, 那么方向必须也是从下到上, 所以就归约到了问题 $[i|j+1|0]$; 如果是从前一行来的, 那么无论前一行是方向是向上还是向下都可以, 所以归约到了问题 $[i-1|j|0]$ 和 $[i-1|j|1]$ (它们两者取最大值).

对于问题 $[i|j|1]$ 也是同理, 可以归约到 $[i|j-1|1]$ 以及 $[i-1|j|0]$ 和 $[i-1|j|1]$ 上.

于是我们的时间复杂度就优化成了 $O(NM)$.

背包问题

01背包问题

有总共 N 个物品, 每个物品都有体积 v 和价值 p 两个参数, 你有一个容积为 V 的背包, 你需要选取若干个物品装入背包, 在总体积不超过容积的前提下, 使得总价值尽量大, 计算这个最大值.

令问题 $[i|j]$ 表示"仅考虑将前 i 个物品中的若干放入背包中, 并且总体积不超过 j 的方案中, 最大总价值是多少". 最后原问题的答案是 $f[N|V]$.

为求解问题 $[i|j]$ 我们仅需把方案按照第 i 个物品是否被放入背包来进行讨论. 如果第 i 个物品放入背包了, 那么我们只剩下 $j-v_i$ 的容积来考虑前 $i-1$ 个物品, 我们希望总价值最大, 所以这个问题就归约到了 $[i-1|j-v_i]$ 上, 这一类的最大值即为 $f[i-1|j-v_i]+p_i$; 如果我们不放, 那么我们仍然剩余 j 的容积来考虑前 $i-1$ 个物品, 于是问题就归约到了 $[i-1|j]$ 上. 综上, 我们可以将 $[i|j]$ 归约到 $[i-1|j]$ 和 $[i-1|j-v_i]$ 上来解决. 时间复杂度为 $O(NV)$.

完全背包

有总共 N 类物品, 每类物品都有体积 v 和价值 p 两个参数(每类物品不设上限, 可以取出任意个), 你有一个容积为 V 的背包, 你需要选取若干个物品装入背包, 在总体积不超过容积的前提下, 使得总价值尽量大, 计算这个最大值.

- Hint: 采用跟01背包一样的问题设计, 然后看看该如何修改问题的归约方式.

为解决问题 $[i|j]$ 我们可以考虑如下情况: 第 i 类物品取还是不取. 如果不取, 那么直接就归约到 $[i-1|j]$; 如果取, 那么至少取一个, 那我们就把这第一个取出来先, 那么还剩下 $j-v_i$ 的容积, 让这 $j-v_i$ 去考虑是否继续取第 i 类物品, 于是问题就归约到了 $[i|j-v_i]$ 上.

多重背包

有总共 N 类物品, 每类物品都有体积 v , 价值 p 和个数 k 三个参数, 你有一个容积为 V 的背包, 你需要选取若干个物品装入背包, 在总体积不超过容积的前提下, 使得总价值尽量大, 计算这个最大值.

如果我们采取与前面一样的问题的设计方式, 我们在解决 $[i|j]$ 的时候还需要枚举第 i 类物品放入多少个, 即枚举 $x \leq k$ 后把 $[i|j]$ 归约到 $[i-1|j-x \cdot v_i]$ 上, 时间复杂度为 $O(V \cdot \sum k_i)$.

多重背包plus

接下来我们提供一种二进制拆分优化的方式,将多重背包归约到 01 背包上. 对于正整数的 k , 我们取最大的整数 L 使得 $\sum_{0 \leq i \leq L} 2^i \leq k$. 之后考虑 $L+2$ 个数, 其中前 $L+1$ 个数为

$$2^0, 2^1, \dots, 2^{L-1}, 2^L$$

记它们分别为 $b_0, b_1, \dots, b_{L-1}, b_L$. 最后一个数为

$$k - \sum_{0 \leq i \leq L} 2^i$$

记之为 b_{L+1} . 我们可以发现, 对于任意 $x \leq k$, 总存在一种从 $\{b_i : i \leq L+1\}$ 中选择若干个数的方法使得选出来的数之和恰好为 x . 于是我们可以把多重背包中每类物品(第 i 类)都根据我们的这种拆分方式拆成 $\log_2(k_i)+1$ 个物品, 其中第 j 个物品的体积是 $b_j \cdot v_i$, 价值是 $b_j \cdot p_i$, 之后我们在这拆分出来的 $\sum \log_2(k_i)+1$ 个物品上跑 01 背包即可. 时间复杂度为 $O(V \cdot \sum \log_2 k_i)$.

树上的动态规划

某大学有 N 个职员, 编号为 $1, 2, \dots, N$. 他们之间有从属关系, 也就是说他们的关系就像一棵以校长 1 为根的树, 父结点就是子结点的直接上司. 现在有个周年庆宴会, 宴会每邀请来一个职员都会增加一定的快乐指数 r_i , 但是呢, 如果某个职员是直接上司来参加舞会了, 那么这个职员就无论如何也不肯来参加舞会了. 请你编程计算, 邀请哪些职员可以使快乐指数最大, 求最大的快乐指数.

令问题 $[i|0/1]$ 为"只考虑是否邀请以 i 为根的子树内的职员, 在 i 职员的状态是 0: "不来" \1: "来"的前提下, 所有邀请方案中快乐指数的最大值", 最终答案是 $\max(f[1|0], f[1|1])$.

假定节点 i 的儿子集合为 son_i , 那么对于问题 $[i|0]$, 既然 i 不来, 那么它的儿子 $j \in \text{son}_i$ 来或者不来都可以, 于是我们只需要对于全部儿子 j 都计算出 $[j|0]$ 和 $[j|1]$ 的答案后再取最大值加在一起就可以了; 对于问题 $[i|1]$, 既然 i 来, 那么它的儿子都不会来, 所以我们需要计算 $[j|0]$.

子集上的动态规划

回顾第一节里头讲的如何用一个数的二进制来表示一个集合的子集. 假定集合 S 的大小为 N , 那么对于一个数 $x < 2^N$, 它的二进制表示中第 i 位为 1 则意味着 S 的第 i 个元素在它的子集 P 中; 若第 i 位为 0 则意味着 S 的第 i 个元素不在它的子集 P 中.

事实上两个数字之间的位运算可以对应一个大集合的子集之间的宏运算(比如按位与就是集合取交, 按位或就是集合取并, 按位异或就是集合取环和...). 对于一个二进制数 x , 假设 X 是它对应的子集, 那么条件 $((x > i) \& i) = 0$ 可以判断第 i 个元素是否在 X 中.

如果你把 $x < 2^N$ 看作一个 01 图案, 那么 $x < i$ 就是将图案上的 1 集体左移 i 位, 而 $x > i$ 则是将图案上的 1 集体右移 i 位.

$00(00001110011010) >> 2 \rightarrow (00000011100110)$

$(00001110011010) << 2 \rightarrow (00111001101000)$

互不侵犯

在 $N \times N$ 的棋盘里面放 K 个国王, 使他们互不攻击, 共有多少种摆放方案. 国王能攻击到它上下左右, 以及左上左下右上右下八个方向上附近的各一个格子, 共 8 个格子.

在这一个题目里我们考虑一行一行地放棋子, 具体的设计问题 $[i|j|s]$ 表示 "仅在前 i 行放入 j 个棋子, 并且第 i 行的棋子摆放的图案为 s 的前提下, 总共有多少种摆放方案", 其中 s 是一个小于 2^N 的数, 第 x 位为 1 则表示这一行的第 x 列放了棋子, 当然 s 所描述的是一行的摆放方案, 所以这一行的棋子是不能相互冲突的, 那么 s 应当满足条件 $\text{peace}(s) := (s < 1) \& s = 0$ 且 $(s > 1) \& s = 0$.

令一个图案 x 中 1 的个数为 $\text{popcount}(x)$, 那么对于问题 $[i|j|s]$ 而言, 我们把所有放置方法按照第 $i-1$ 行的图案来分类, 假定第 $i-1$ 行的图案是 x' , 那么 x' 是一个合法的图案当且仅当满足如下的 2 个条件

1. $((s < 1) | s | (s > 1)) \& x' = 0$.
2. $\text{peace}(x')$

在满足了这些条件后, 这一类摆放方案相当于 $[i-1|j-\text{popcount}(s)|x']$ 中的摆放方案再在第 i 行按照 s 来放棋子, 所以这一类的方案数就是 $[i-1|j-\text{popcount}(s)|x']$, 因此我们只需要计算出全体满足条件的 x' 的 $[i-1|j-\text{popcount}(s)|x']$ 后再将答案加起来就可以得到 $[i|j|s]$.

后记

去年的课程设计的本意是希望向听众传达动态规划的思想内核(至少我是这么理解的): 一个问题如何归约到子问题上, 这是我作为一个尤其钟意动态规划的OIer以及ACMer经过多年学习和尝试所得到的一个可能没有那么 trivial 的理解, 我不希望大家只是把动态规划当作一种方法或者解题的套路来学习而抛却了它的思想本质(我本人就是在这种工具主义的思想走了很长的弯路, 付出了很多代价), 事实上在我遇到过的好多十分困难的动态规划问题时(我还没有遇到过的模型), 正是这种思想能够指引我找到解法. 然而市面上大多数的教材与课程只是教会大家各种动态规划的模型, 我想打破这种局面. 为此, 在去年课程中, 我刻意地设计了比较陌生的题意和背景, 之后尝试提示大家往如何把原问题归约到子问题上去思考, 从而能够更好的理解动态规划这类方法的本质. 然而事实上我忽略了大家作为初学者对于一些技术, 方法是陌生的这一事实, 所以就会出现很多我认为讲清楚了而大家实际上是不清楚的情况. 今年我改变了原来的课程结构, 在开头引入了一些简单的动态规划问题来帮助大家建立基本的动态规划思想方法, 在此基础上再逐步展开动态规划这棵大树, 希望能有更好的效果.

然而想要熟练地使用动态规划来解决我们遇到的问题不仅需要知道什么是动态规划, 更需要多多了解动态规划的实践方法: 如何设计状态, 如何转移, 如何优化. 因此要多加练习, 我在此推荐洛谷的题单: [【动态规划】普及~省选的dp题](#).

bonus

有一个 N 个节点的环, 第 i 个点和第 $(i \bmod N) + 1$ 个点相连, 每个节点上都有一个权值 a , 保证所有点的权值之和为负, 定义一个路径上的最大值是它经过的所有节点的权值之和(重复经过算多次) 请你计算从某个节点出发, 权值最大的路径的权值是多少.

如果我们令问题 $[i]$ 表示 "以 i 为终点的路径中权值最大的路径的权值是多少", 那么我们可能需要考虑路径中 i 前那个节点是谁, 它显然是 $i-1$ 或者 $i+1$, 于是我们似乎就能够把问题归约到 $[i-1]$ 和 $[i+1]$. 但是为了计算 $[i-1]$ 和 $[i+1]$ 我们又需要去计算 $[i-2], [i], [i+2]$, 于是我们又回到了问题 $[i]$ 上.

这就是所谓的环结构问题, 在遇到这些问题时我们需要考虑借助一些题目给出的额外的信息来破坏环的结构, 或者重新设计问题, 来使得我们能够找到一个计算问题的顺序.

课上的草稿

给定两个长度为 N 的正整数序列 $\langle a_i \rangle, \langle b_i \rangle$, 求最长的公共上升子序列的长度, 设计一个时间复杂度不超过 $O(N^3)$ 的算法.

- Hint: 考虑设计问题 $[x|y]$ 为 "前缀 $\langle a_1, \dots, a_x \rangle$ 与 $\langle b_1, \dots, b_y \rangle$ 的以 b_y 为结尾的最长公共上升子序列的长度".

为了解决 $[x|y] \rightarrow$ 分类

1. a_x 不出现在最长公共上升子序列中的情形 $\rightarrow [x-1|y]$
2. a_x 出现在最长公共上升子序列中的情形 $a_x = b_y$

在 $a_1 \dots a_{x-1}; b_1 \dots b_{y-1}$ 中选一个最长的公共上升子序列结尾元素小于 b_y

枚举以 b_k 为结尾: 枚举所有的 $k < y$ 且 $b_k < b_y$ 的 k , 之后规约 $[x-1|k]$

设有 N 行 M 列的方格图, 每个方格中都有一个整数. 现有一只小熊, 想从图的左上角走到右下角, 每一步只能向上, 向下或向右走一格, 并且不能重复经过已经走过的方格, 也不能走出边界. 小熊会取走所有经过的方格中的整数, 求它能取到的整数之和的最大值. 设计一个 $O(N^2M)$ 复杂度的算法.

同样的问题, 给出一个 $O(NM)$ 的算法.

- Hint: 有没有更好的方法来刻画小熊的路径呢? 用更通俗的话来解释 "第 i 列的终点的行数必须等于第 $i+1$ 列的起点的行数" 这个限制.

形式化的来讲, 小熊走的一条完整的路径可以被切分成 M 段, 其中第 i 段是在第 i 列上连续的若干行, 并且第 i 列的终点的行数必须等于第 $i+1$ 列的起点的行数.

有总共 N 类物品, 每类物品都有体积 v 和价值 p 两个参数(每类物品不设上限, 可以取出任意个), 你有一个容积为 V 的背包, 你需要选取若干个物品装入背包, 在总体积不超过容积的前提下, 使得总价值尽量大, 计算这个最大值.

- Hint: 采用跟01背包一样的问题设计, 然后看看该如何修改问题的归约方式.

$[i|j]$ 表示仅考虑将前 i 个物品中的若干放入背包中, 并且总体积不超过 j 的方案中, 最大总价值是多少

如果 $x \leq \sum_{i \leq L} 2^i$

如果 $x > \sum$ -> 一定选取 b_{L+1} 剩下的就是在 $2^0, 2^1, \dots, 2^L$ 里头选出 $x - b_{L+1}$

$x \leq k \Rightarrow x \leq b_{L+1} + \sum$

$[i|j|0/1]$ i 是列, j 是行

第一层循环 i 从小到大

$[i|j|0] \rightarrow [i|j+1|0], [i-1|j|0/1]$

$[i|j|1] \rightarrow [i-1|j|0/1], [i|j-1|1]$

```
for(int i=1; i<=m; ++i){
    //计算问题 0
    for(int j=n; j>=1; --j){
        解决问题 [i|j|0]
    }
    //计算问题 1
    for(int j=1; j<=n; ++j){
        解决问题 [i|j|1]
    }
}
```